Low Level, Top Floor: Pricing Every Level of a C++/WebAssembly Application

Dao Huynh
University of California, Los Angeles
daohuynh@g.ucla.edu
+1 657 642 9500

Introduction

This poster reports on a C++23 application compiled to a single WebAssembly binary, live in production, in which every level above the GPU is written in C++: the render backend under Dear ImGui, the focus policy, the text inputs, the screen layout, the persistence, the theming, the audio. There is no JavaScript application layer and no framework above the engine boundary, and what the poster reports is the price of that decision: what each level cost, with the platform's price for the same level beside it.

An application is a building, and the ground floor is the GPU. Every floor above it is a level where the platform offers an answer and you are free to write your own; this one is owned all the way up. It was specified across eighty pages before the first line was written, implemented in six weeks into fourteen zones behind sixteen sealed interface headers, and shipped with every specified feature intact; the source runs 33,780 lines against 11,867 lines of tests. The bill is the poster.

Relevance

Every project that ships C++ to a browser answers a question it rarely notices asking. How far up does the C++ go? Convention answers for most of them, and the answer is a good one; compile the engine, the solver, the codec, write the application layer in TypeScript above it, and leave the DOM the floors it was designed for. A good convention inherited is still a convention inherited. A project that follows one has priced exactly one side of its own boundary, because pricing a floor means building it twice, and no project builds the side it rejected.

This one built every floor. That was a decision carried further up than convention carries it, and it produced the condition nobody funds on purpose: a complete price list from the metal to the top floor, in one codebase, under one specification, by one author. The list is what I bring. The room has argued about this boundary for as long as Emscripten has existed; the argument has never had a bill.

Discussion

The instrument. A price is only a price if the work was competent, so the calibration comes first. The specification preceded the implementation and was implemented to completion. The fourteen zones sit behind sixteen contract headers sealed before any zone was written and never edited afterward; the include graph across those headers is acyclic by construction, verified by topological sort at ten edges and a maximum depth of three. The test suite is 11,867 lines and compiles to native binaries that run without a browser. The numbers below are what floors cost when someone owns them deliberately.

Why the bill has to be counted. Asked which zone held the decision mathematics, I named the wrong one. I pointed at 1,604 lines and called it the engine; 482 of those lines grade a submitted answer, and the remainder render input boxes and bind keys. The mathematics the product exists to perform is 665 lines, in a zone I did not name, and I wrote all of it. The rent is invisible from inside the building, to the tenant, six weeks after signing. That is the case for counting.

The rule. Every file in the source tree is classified once. *Domain* is the poker decision mathematics. *Platform* is any file whose primary job is substrate the browser hands you directly: focus and keyboard navigation, text input, scrolling, layout, animation timing, theming, asset fetch and decode, audio plumbing, persistence plumbing, and the glue between wasm and the page. *Inherent* is what you would write in any language for any target: scenario generation, tutorial content, game and economy semantics. The rule is published so it can be attacked, and the script that applies it ships with the poster. Lines of code do one narrow job in this table; they measure surface owned, which is surface maintained. They are not asked to measure effort, difficulty, or time.

Zone	Total	Domain	Platform	Inherent
bridge	5,889	0	5,378	511
settings	3,996	0	3,338	658

modal	3,869	0	3,869	0
screens	3,682	0	3,682	0
persistence	3,605	0	3,426	179
render	2,039	0	2,039	0
tutorial	1,970	0	1,098	872
engine	1,675	665	0	1,010
math	1,604	482	1,122	0
backbone	1,462	0	1,071	391
audio	1,214	0	1,112	102
assets	1,127	0	1,127	0
theme	746	0	746	0
temporal	419	93	326	0
animations	401	0	401	0
easter_egg	64	0	0	64
Total	**33,762**	**1,240**	**28,735**	**3,787**
Share of source		**3.7%**	**85.1%**	**11.2%**

The bill, by zone, in lines of source. Twenty-three lines of floor for every line of poker. The classification walks the sixteen zone directories; the eighteen lines of the tree that sit outside any zone are left unclassified.

The figure has a floor and a ceiling, and both belong in the table's company. Roughly 17,846 lines are pure substrate, of which a DOM application writes approximately none; 10,889 lines are interface chrome that a DOM application still writes, more cheaply, in markup and CSS. The recoverable share is smaller than 85 percent. The proportion holds.

What the platform charges for the same floor. Five floors carry most of the bill, and each one has a tenant already living in it downstairs.

Floor	Mine	What the platform rents it for
settings dialog	2,946	<form>, <input>, <select>, native scroll
leaderboard and modal framework	1,765	<dialog>, <table>, native scrollbars
tutorial overlay	1,072	positioned div, box-shadow spotlight
math text inputs and keybinds	983	<input type="number">; caret, IME, clipboard
focus policy	588	tabindex, :focus-visible

The focus floor is the one with a measured price on both sides. My focus system is 588 lines: a registry of stops, a per-frame reconciliation of the application's belief against the library's, and a context stack that isolates modal focus so that a background element cannot be addressed while a modal is open. It runs in 2.5 ns on a steady frame and allocates nothing. I audited it against Dear ImGui's own keyboard navigation by driving both headlessly through the same sixteen behaviors. Twelve came back native. Three came back as a few lines of key ownership. One came back a difference in model; the library reports a consumed key through ownership, where mine reports it through a return value. None came back impossible. The browser gives all sixteen for an attribute and a pseudo-class. That floor cost 588 lines, and I know what it goes for elsewhere.

Three costs are stacked inside every number above, and the poster separates them where the source allows. Some of the bill is the language. Some of it is the paradigm, since an immediate-mode canvas has no retained widget to hang behavior on. Some of it is the ecosystem, since a framework would have compressed the chrome in any language. One ratio, three causes; untangling them is named below as work outstanding.

What the ownership bought. The engine is pure and reconstructable from a 64-bit seed, and that determinism is why 11,867 lines of tests compile native and run in continuous integration with no browser, no headless driver, and no flake. It is the one thing the convention cannot easily hand back, and it is real. It is also some two thousand lines of advantage standing against twenty-eight thousand lines of floor. The same product with its boundary drawn lower would have kept the determinism and deleted most of the substrate. I did not draw it lower, which is why the list exists.

Shape. The ratio is a property of the product and not of the language. A poker trainer is a large interface over a small mathematical core, and any implementation of that shape pays a steep floor-to-substance ratio; the conventional one pays it in someone else's code. Invert the shape, put a compiler or a physics engine or a codec at the center, and the same rule

returns a different answer that is equally correct. The number on this poster is mine. The rule is portable. A reader who runs it over their own tree gets their own bill, and their own answer about how far up their C++ should go.

Completion Status

Complete. The application is specified, implemented, tested, and live in production; every feature in the specification shipped. The classification is complete across all 244 source files and the script that produced it is re-runnable. The navigation audit is complete and source-cited against Dear ImGui 1.91.9b.

Limits. This is one codebase with no control; nothing here shows that the method caused the outcome. Lines of code measure surface, and the poster asks them for nothing else. The platform's price is measured at the focus floor and estimated at the others. The classification of the render and screens zones is the most aggressive call in the table, and it is flagged as such in the data.

Before September. The counterfactual gets a measurement. The settings dialog, the largest single line item at 2,946 lines, will be reimplemented as a DOM overlay over the canvas, using the same bridge the Terms and Privacy documents already ride, with the feature set held constant and both prices real. The three stacked costs get separated into language, paradigm, and ecosystem, so a reader can take the axis that applies to them. Two axes independent of line count get mined from six weeks of commit history: session-clustered time per zone, and defect density per zone from fix commits. Either may contradict the table, and either will be published if it does. A second classification pass under a rule chosen against me will bound the estimate from the other side. The poster itself will carry the zone map, the bill, and a QR code to the running application.

Supporting Material

Live application: **poker-trainer-pearl.vercel.app**
Source: **github.com/daohhuynh/poker-trainer**
The classifier and its rule, the Dear ImGui navigation audit, and the raw measurement data accompany the source.